



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Operating Systems – CS234AI

LAB EXPERIENTIAL LEARNING

Process Scheduler

submitted by

Amritesh Sahu

1RV23CD006

Apoorv Kumar Verma

1RV23CD012

Under the Guidance of

Assistant Professor

Dr. Sneha M

Department of CSE, RVCE, Bengaluru



RV College of Engineering[®]

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

PROJECT CERTIFICATE

This is to certify that Amritesh Sahu and Apoorv Kumar Verma has satisfactorily completed the course of Experiential Learning in Operating Systems by the Department of Computer Science and Engineering – Cyber Security during the year 2024-25.

NAME

Amritesh Sahu

APOORV KUMAR VERMA

USN

1RV23CD006

1RV23CD012

SIGNATURE OF THE STAFF

HEAD OF DEPARTMENT

TABLE OF CONTENTS

SL.NO	CONTENTS	PAGE. NO
1.	INTRODUCTION	4
2.	METHODOLOGY	4
3.	RESULTS	6
4.	OUTPUT	7
5.	CONCLUSION	8

1. Introduction CPU scheduling is a critical function of an operating system that determines how processes

are assigned to the CPU, directly impacting system performance and resource utilization. The choice of scheduling algorithm influences factors such as response time, throughput, and CPU efficiency.

This project focuses on two commonly used scheduling algorithms: **Round Robin (RR)** and **Priority Scheduling**. Round Robin ensures fair CPU allocation by providing each process with an equal time slice, while Priority Scheduling assigns processes a priority, allowing high priority tasks to be executed first. The aim of this project is to simulate and compare these algorithms under different system conditions to evaluate their performance, efficiency, and fairness.

2. Background 2.1. CPU Scheduling CPU scheduling determines how processes are assigned CPU time in a multi-tasking environment. It is vital for maximizing CPU utilization and system responsiveness. Scheduling algorithms are designed to balance the competing goals of fairness (equal CPU time for all processes) and efficiency (minimizing waiting times and maximizing throughput). Key performance metrics include:

- **CPU Utilization:** Percentage of time the CPU is active.
- **Throughput:** Number of processes completed in a given time.
- **Waiting Time:** Time a process spends in the ready queue before execution.
- **Response Time:** Time from process arrival to the first response.
- **Turnaround Time:** Total time from process arrival to completion.

2.2. Round Robin (RR) Scheduling

Round Robin is a **pre-emptive scheduling algorithm**, designed to ensure fairness by allocating each process a fixed time slice (or **quantum**). If a process doesn't complete within its allocated time, it is moved to the back of the ready queue, and the next process gets its turn. This continues cyclically until all processes are completed.

Advantages:

- Fair and prevents starvation.
- Simple to implement.

Disadvantages:

- Frequent context switching increases overhead.
- Not ideal for processes with varying burst times.

2.3. Priority Scheduling Priority Scheduling assigns a priority to each process, and the process with the highest priority gets CPU time first. This can be either **pre-emptive** (where a higher-priority process pre-empts a lower-priority process) or **non-pre-emptive** (where once a process starts executing, it completes before any higher-priority process is allowed to run).

Advantages:

- Efficient for systems with tasks that require immediate attention (e.g., real-time systems).
- Reduces waiting time for high-priority processes.

Disadvantages:

- Starvation: Lower-priority processes may be indefinitely delayed if higher-priority tasks keep arriving.
-

3. Problem Statement In a multi-programmed environment, efficient CPU scheduling is essential to optimize

performance and ensure fair resource allocation. This project aims to simulate and compare two commonly used scheduling algorithms: **Round Robin (RR)** and **Priority Scheduling**. The focus is on understanding how these algorithms perform under different system loads, and how they impact key performance metrics such as CPU utilization, throughput, response time, and turnaround time.

4. Methodology

4.1. Algorithm Analysis

- **Round Robin (RR):** Each process is allocated a fixed time quantum. Once the quantum expires, the process is moved back to the queue, and the next process is given CPU time. This process continues until all processes finish.
Priority Scheduling: Each process is assigned a priority value. The CPU is allocated to
 - the process with the highest priority. In case of a tie, additional rules like **First Come First Serve (FCFS)** or Round Robin may be used to break the tie.

4.2. Simulation Design

Various scenarios were simulated to test the performance of each algorithm:

- **Round Robin:** The time quantum was varied to examine its impact on response time and efficiency. The system handled processes with varying burst times to determine how well RR maintains fairness.
- **Priority Scheduling:** Processes were assigned random priorities, and the system was tested with varying workloads to assess how the algorithm prioritizes tasks.

4.3. Performance Metrics

To evaluate the performance of both algorithms, the following metrics were used:

- **CPU Utilization:** The percentage of time the CPU is in use.
- **Throughput:** The number of processes completed in a given time period.
- **Waiting Time:** The time a process spends in the ready queue before execution.
- **Response Time:** The time from when a process arrives to when it starts execution.
- **Turnaround Time:** The total time from the arrival of a process to its completion.

4.4. Development Tools

To simulate the algorithms and present the results, the following technologies were used:

- **HTML/CSS/JavaScript:** For simulating the algorithms and calculating the performance metrics. For building the user interface.
 - **Web Interface:** Users can input process parameters such as arrival time, burst time, priority, and quantum, and visualize the results.
-

5. Implementation

5.1. Round Robin Scheduling Implementation

The Round Robin scheduling algorithm was implemented using the following steps:

- Maintain a list of processes with attributes such as Process ID, arrival time, burst time, and remaining time.
- Each process is allocated a time quantum. If the process doesn't complete within the quantum, it is placed at the end of the queue for the next round.

After each cycle, the waiting time, turnaround time, and response time are

- recalculated.

-

5.2. Priority Scheduling

Implementation in Priority

Scheduling:

- Each process is assigned a priority. Processes with higher priority values are executed before lower-priority ones.

If two processes have the same priority, a secondary rule (like FCFS or RR) is used to break the tie.

- As processes are executed, metrics like waiting time and turnaround time are updated.

5.3. Web-Based Simulation

Interface The web interface

allows users to:

- Input process details like arrival time, burst time, priority, and quantum (for RR).
- View a graphical timeline that shows the execution order of processes.
- View performance metrics in real-time, with dynamic updates based on the simulation results.

5.4. Testing and Validation

The algorithms were tested using various combinations of input values, such as:

- High and low time quantum for Round Robin.
- Equal priority processes in Priority Scheduling.
- Edge cases like very short or very long burst times.

5.5. Optimization and Performance Analysis

- **Optimization for RR:** Dynamic adjustment of the time quantum based on the average burst time of processes to minimize CPU idle time.
 - **Optimization for Priority Scheduling:** Adaptive priority adjustments to prevent starvation of lower-priority tasks.
-

-

6. Results and Discussion

6.1. Simulation Results

Tables, graphs, and charts were used to display the simulation results. The following comparisons were made:

- **CPU Utilization:** Higher CPU utilization was achieved in Priority Scheduling for tasks with high priority.
- **Throughput:** Round Robin had better throughput in environments with equal priority tasks.
- **Waiting Time and Response Time:** Round Robin had more balanced waiting times, while Priority Scheduling reduced waiting time for high-priority tasks.

6.2. Performance Comparison

Round Robin: Effective in ensuring fairness and minimizing starvation, but can have higher overhead due to context switching.

- **Priority Scheduling:** Effective for real-time tasks but susceptible to starvation of lower priority processes.
-

7. Conclusion

Both Round Robin and Priority Scheduling algorithms have their strengths and weaknesses. Round Robin ensures fairness but can lead to increased overhead. Priority Scheduling prioritizes critical tasks but may cause starvation for lower-priority processes. This project demonstrates that the choice of scheduling algorithm depends on the system's requirements, workload characteristics, and the need for fairness versus efficiency.

8. References

- Stallings, W. (2018). *Operating Systems: Internals and Design Principles*. 9th Edition. Pearson.
 - Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts*. 10th Edition. Wiley.
-

9. VISUALS

CPU Scheduling

Algorithm
FCFS

ADD
PROCESS

SIMULATE

Process 1:

Arrival Time
0

Burst Time
5

REMOVE

Process 2:

Arrival Time
2

Burst Time
3

REMOVE

Process 3:

Arrival Time
4

Burst Time
1

REMOVE

Process 4:

Arrival Time
5

Burst Time
2

REMOVE

Process	Arrival Time	Burst Time	Start Time	End Time	Waiting Time	Turnaround Time
P1	0	5	0	5	0	5
P2	2	3	5	8	3	6
P3	4	1	8	9	4	5
P4	5	2	9	11	4	6

Average Waiting Time: 2.75
Average Turnaround Time: 5.5



CPU Idle Time: 0 units
Completion Time: 11 units

CPU Scheduling

Algorithm
SJF

ADD PROCESS

SIMULATE

Process 1:	Arrival Time 0	Burst Time 5	REMOVE
Process 2:	Arrival Time 2	Burst Time 3	REMOVE
Process 3:	Arrival Time 4	Burst Time 1	REMOVE
Process 4:	Arrival Time 5	Burst Time 2	REMOVE

Process	Arrival Time	Burst Time	Start Time	End Time	Waiting Time	Turnaround Time
P1	0	5	0	5	0	5
P2	2	3	5	8	3	6
P3	4	1	8	9	4	5
P4	5	2	9	11	4	6

Average Waiting Time: 2.75

Average Turnaround Time: 5.5



CPU Idle Time: 0 units

Completion Time: 11 units

CPU Scheduling

Algorithm
RR

Quantum
4

ADD PROCESS

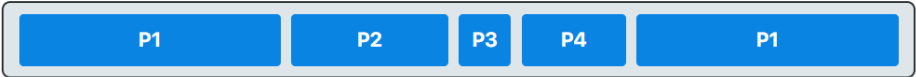
SIMULATE

Process 1:	Arrival Time 0	Burst Time 5	REMOVE
Process 2:	Arrival Time 2	Burst Time 3	REMOVE
Process 3:	Arrival Time 4	Burst Time 1	REMOVE
Process 4:	Arrival Time 5	Burst Time 2	REMOVE

Process	Arrival Time	Burst Time	Start Time	End Time	Waiting Time	Turnaround Time
P1	0	5	0	4	0	4
P2	2	3	4	7	2	5
P3	4	1	7	8	3	4
P4	5	2	8	10	3	5
P1	0	5	10	11	10	11

Average Waiting Time: 3.6

Average Turnaround Time: 5.8



CPU Idle Time: 0 units

